



A PROJECT REPORT ON

Automation Testing Project on OWASP Juice Shop

SUBMITTED BY:

Modi Hetalben Smithkumar

## **ABSTRACT**

The purpose of this project is to design and develop an automated testing framework for the OWASP Juice Shop web application using Selenium WebDriver with Java.

OWASP Juice Shop is a deliberately vulnerable e-commerce application developed to demonstrate common security and functional issues found in modern web applications. This project focuses on automating key functional modules of the application to ensure reliability, accuracy, and efficiency in the testing process.

A Hybrid Automation Framework has been implemented by integrating Page Object Model (POM), Data-Driven, and Keyword-Driven approaches. TestNG is used as the testing framework to manage test execution, annotations, and reporting, while Maven is utilized for project build and dependency management. Test data is externalized using Excel files to enhance reusability and maintainability of test scripts.

The automation suite covers major functionalities such as user registration, login, product search, add-to-cart operations, and basic validations. The framework supports cross-browser execution, reusable utilities, screenshot capture on failure, and detailed test reporting. This automated solution reduces manual effort, improves test coverage, and ensures consistent test execution.

Overall, this project demonstrates an efficient and scalable automation testing solution for web applications and provides practical exposure to real-time automation frameworks used in the software testing industry.

## TABLE OF CONTENTS

1. **Introduction**
  - 1.1 Project Overview
  - 1.2 Purpose of the Project
  - 1.3 Scope of the Project
  - 1.4 Objectives
2. **Application Overview**
  - 2.1 About OWASP Juice Shop
  - 2.2 Application Architecture
  - 2.3 Key Features
3. **Tools & Technologies Used**
  - 3.1 Selenium WebDriver
  - 3.2 Java
  - 3.3 TestNG
  - 3.4 Maven
  - 3.5 Eclipse IDE
4. **Automation Framework Design**
  - 4.1 Framework Overview
  - 4.2 Hybrid Framework Architecture
  - 4.3 Page Object Model (POM)
  - 4.4 Data-Driven Approach
  - 4.5 Keyword-Driven Approach
5. **Project Structure**
  - 5.1 Folder Structure
  - 5.2 Description of Packages
  - 5.3 Configuration Files
6. **Test Planning**
  - 6.1 Test Strategy
  - 6.2 Test Scenarios
  - 6.3 Test Case Design
  - 6.4 Test Data Management
7. **Test Environment Setup**
  - 7.1 Hardware Requirements
  - 7.2 Software Requirements
  - 7.3 Browser Configuration

## **8. Test Execution**

8.1 Test Execution Flow

8.2 Smoke Testing

8.3 Regression Testing

8.4 TestNG Execution

## **9. Test Cases**

9.1 Login Module

9.2 Registration Module

9.3 Product Search Module

9.4 Cart Module

## **10. Defect Management**

10.1 Defect Life Cycle

10.2 Defect Report Format

10.3 Sample Defects

## **11. Test Reports**

11.1 TestNG Reports

11.2 Execution Summary

## **12. Challenges & Limitations**

12.1 Challenges Faced

12.2 Limitations of Automation

## **13. Conclusion**

## LIST OF FIGURES

| Figure No.  | Figure Title                             |
|-------------|------------------------------------------|
| Figure 1.1  | OWASP Juice Shop Home Page               |
| Figure 2.1  | OWASP Juice Shop Login Page              |
| Figure 2.2  | OWASP Juice Shop Registration Page       |
| Figure 3.1  | Automation Testing Life Cycle            |
| Figure 4.1  | Hybrid Automation Framework Architecture |
| Figure 4.2  | Page Object Model (POM) Design           |
| Figure 5.1  | Project Folder Structure                 |
| Figure 6.1  | Test Execution Flow Diagram              |
| Figure 7.1  | Sample Test Case Format                  |
| Figure 8.1  | Selenium WebDriver Execution             |
| Figure 9.1  | TestNG Execution Report                  |
| Figure 10.1 | Sample Defect Report                     |

## **CHAPTER : 1 Introduction**

### **PROJECT SUMMARY**

This project focuses on the automation testing of the OWASP Juice Shop web application using Selenium WebDriver with Java. OWASP Juice Shop is a modern, intentionally vulnerable e-commerce application that provides a realistic platform for performing functional and security-related testing. The primary goal of this project is to automate critical functional workflows to improve test efficiency, accuracy, and repeatability.

A Hybrid Automation Framework has been designed and implemented by combining the Page Object Model (POM), Data-Driven, and Keyword-Driven approaches. TestNG is used as the test management framework to control test execution, grouping, and reporting, while Maven is utilized for dependency management and build automation. External test data is maintained in Excel files, enabling easy updates and improved test scalability.

The automation suite covers major functional modules including user registration, login, product search, add-to-cart, and basic cart management operations. The framework supports reusable components, centralized configuration management, screenshot capture on test failure, and detailed test reports for effective analysis.

By implementing this automation solution, manual testing effort is significantly reduced, test coverage is enhanced, and consistent results are achieved across multiple test executions. This project demonstrates practical exposure to industry-standard automation practices and provides a scalable solution suitable for real-time web application testing.

## PURPOSE

The purpose of this project is to develop an automated testing solution for the OWASP Juice Shop web application using Selenium WebDriver with Java. The project aims to validate the core functional features of the application by automating repetitive and time-consuming test cases, thereby reducing manual testing effort and improving testing efficiency.

This project also aims to design and implement a Hybrid Automation Framework that ensures reusability, scalability, and maintainability of test scripts. By integrating Page Object Model (POM), Data-Driven, and Keyword-Driven approaches, the framework enables structured test execution and easy management of test data and test cases.

Additionally, the project provides hands-on experience with real-time automation tools and best practices followed in the software testing industry. It helps in understanding automated test execution, reporting, defect identification, and validation of web application behavior in different test scenarios.

Overall, the purpose of this project is to demonstrate how automation testing can improve software quality, ensure consistent results, and support faster and more reliable application testing.

## SCOPE

The scope of this project is limited to the automation testing of key functional modules of the OWASP Juice Shop web application using Selenium WebDriver with Java. The project focuses on validating the core functionalities that are most critical to end users, ensuring that the application behaves as expected under different test conditions.

The automation scope includes user registration, login functionality, product search, add-to-cart operations, and basic cart management features. Test cases are designed and executed using a Hybrid Automation Framework that incorporates Page Object Model (POM), Data-Driven, and Keyword-Driven approaches to improve test reusability and maintainability.

This project covers automated execution on a web browser environment and includes functional validation, UI interaction testing, and basic input validation. The scope also includes test data management using external Excel files, screenshot capture on test failures, and generation of detailed test execution reports.

Out of scope for this project are advanced security vulnerability exploitation, performance testing, and backend/API testing. The project is intended for educational and demonstration purposes to showcase automation testing concepts and framework design rather than complete security assessment of the application.

## REPORT OUTLINE

- **Chapter 1: Introduction**

This chapter provides an overview of automation testing, the purpose of the project, scope, and objectives of automating the OWASP Juice Shop application.

- **Chapter 2: Application Overview**

This chapter describes the OWASP Juice Shop application, its features, and its relevance for automation and testing practice.

- **Chapter 3: Tools and Technologies**

This chapter explains the tools and technologies used in the project such as Selenium WebDriver, Java, TestNG, Maven, and Eclipse IDE.

- **Chapter 4: Automation Framework Design**

This chapter details the Hybrid Automation Framework architecture, Page Object Model, data-driven approach, and project structure.

- **Chapter 5: Test Planning and Design**

This chapter covers test strategy, test scenarios, test case design techniques, and test data management.

- **Chapter 6: Test Environment Setup**

This chapter describes the hardware and software requirements along with browser and environment configuration.

- **Chapter 7: Test Execution**

This chapter explains the test execution process, automation flow, and execution strategies such as smoke and regression testing.

- **Chapter 8: Test Cases and Results**

This chapter presents the automated test cases executed for different modules along with execution results.

- **Chapter 9: Defect Management**

This chapter explains the defect lifecycle, defect reporting format, and sample defects identified during testing.

- **Chapter 10: Test Reports and Analysis**

This chapter includes TestNG reports, screenshots, and execution summaries.

## **EXISTING SYSTEM**

In the existing system, testing of the web application is mainly carried out using manual testing methods, where testers execute test cases by interacting with the application through a browser. This process is time-consuming, repetitive, and highly dependent on human effort, making it prone to errors and inconsistencies. Regression testing becomes difficult whenever application changes occur, as all test cases must be re-executed manually. Due to limited time and resources, test coverage is often restricted, and maintaining test records and reports is challenging. Overall, the existing system is inefficient for testing modern web applications that require frequent updates and reliable results.

## **PROBLEM IN EXISTING SYSTEM**

The existing manual testing system faces several problems such as high time consumption, increased human effort, and a higher chance of errors during test execution. Repeating the same test cases for regression testing is inefficient and slows down the development process. Manual testing also provides limited test coverage and makes it difficult to execute large numbers of test cases within a short time. Additionally, maintaining test data, tracking defects, and generating detailed reports require extra effort, leading to inconsistencies and delayed software releases.

## **PROBLEM SOLUTION**

The problems of the existing manual testing system are addressed by implementing an automated testing solution using Selenium WebDriver with Java. A Hybrid Automation Framework is designed to automate repetitive and time-consuming test cases, reducing human effort and minimizing errors. Automation enables faster execution, improved test coverage, and efficient regression testing whenever application changes occur. External test data management, reusable test components, automated reporting, and screenshot capture on failures further enhance testing efficiency and reliability. This solution ensures consistent results, reduces testing time, and improves overall software quality.

## Chapter 2: Application Overview

### 2.1 Introduction

The **OWASP Juice Shop** is an intentionally vulnerable web application developed by the Open Web Application Security Project (OWASP) to teach and practice web security testing. It simulates a real-world e-commerce application, providing functionalities such as product browsing, search, user authentication, and cart management.

This application is widely used in **automation testing, penetration testing, and security learning** due to its full coverage of **OWASP Top 10 vulnerabilities**.

### 2.2 Purpose of the Application

The purpose of using OWASP Juice Shop for this project is:

1. To provide a **realistic web application** for practicing **automation testing**.
2. To allow testers to **create, execute, and validate test scripts** in a controlled environment.
3. To simulate **real-world user interactions**, including login, signup, product search, and cart management.
4. To introduce testers to **common web vulnerabilities**, although in this project the focus will be on **functional automation testing**, not security exploitation.

## 2.3 Application Features

The Juice Shop application includes a variety of features that can be automated and tested. Key features include:

| Feature                   | Description                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------------------|
| User Registration & Login | Users can register, login, and manage their accounts.                                           |
| Product Browsing & Search | Users can search for products, view product details, and filter items.                          |
| Shopping Cart             | Users can add or remove products, view cart details, and proceed to checkout.                   |
| Order Management          | Users can place orders and track their purchase history.                                        |
| Feedback & Ratings        | Users can submit product feedback and ratings.                                                  |
| Security Challenges       | Contains intentionally vulnerable scenarios like SQL Injection, XSS, and broken access control. |
| Score Board               | Tracks user progress for challenges (useful for learning and testing).                          |

## 2.4 Relevance for Automation Testing

OWASP Juice Shop is particularly relevant for **automation testing practice** for the following reasons:

1. **Web-Based UI** – The application is browser-based, making it ideal for **Selenium WebDriver** automation.
2. **Multiple Modules** – Functionalities like login, signup, search, and cart provide **multiple test scenarios**.
3. **Dynamic Data Handling** – Automation can include **data-driven testing** with Excel or CSV files.
4. **Realistic Interaction Flow** – The application mimics a real e-commerce workflow, providing a **comprehensive test environment**.
5. **Reporting & Verification** – Testers can validate **success/failure conditions**, take screenshots, and generate reports for automation practice.

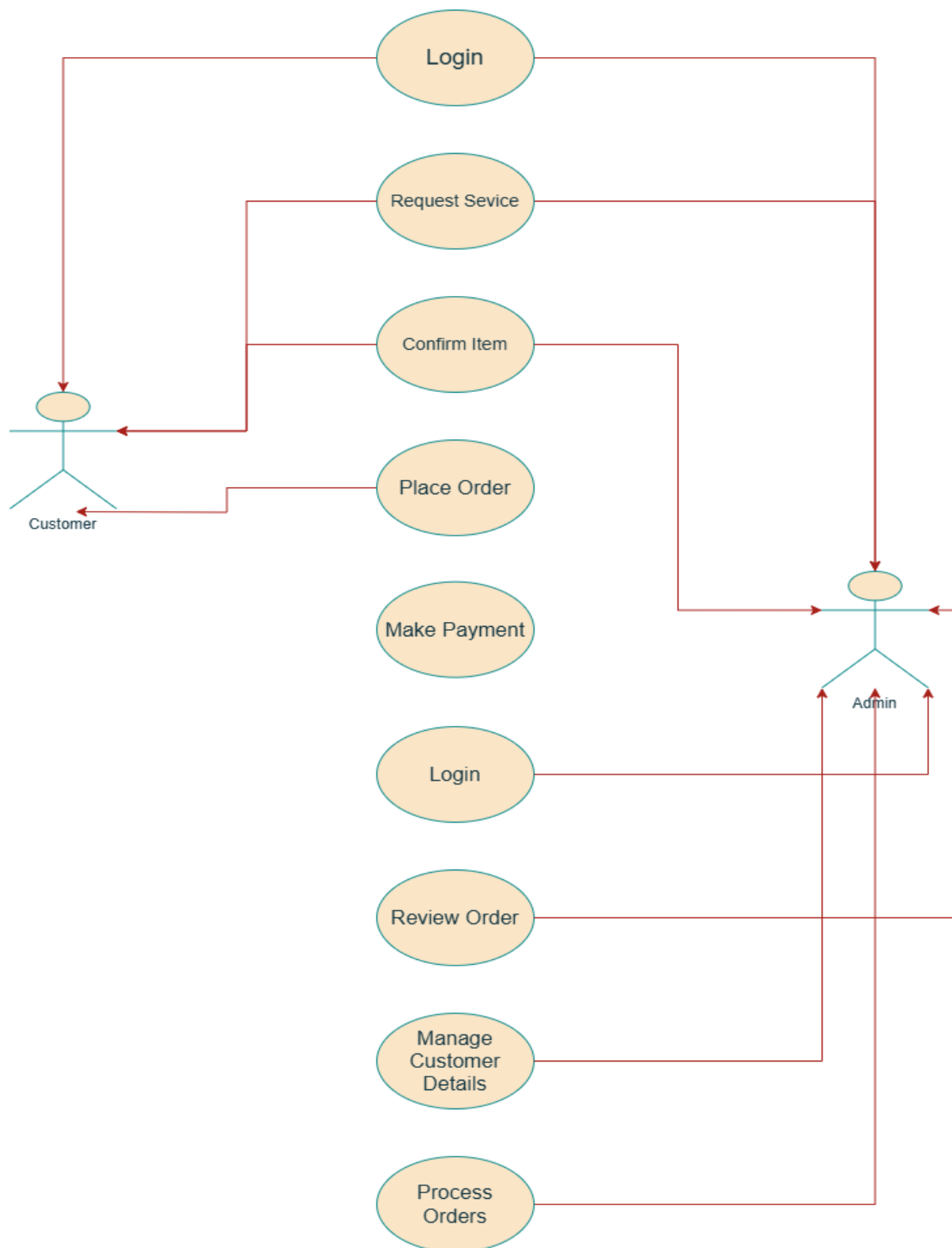
## 2.5 Application Access

The OWASP Juice Shop can be accessed through:

**Demo URL:** <https://demo.owasp-juice.shop/>

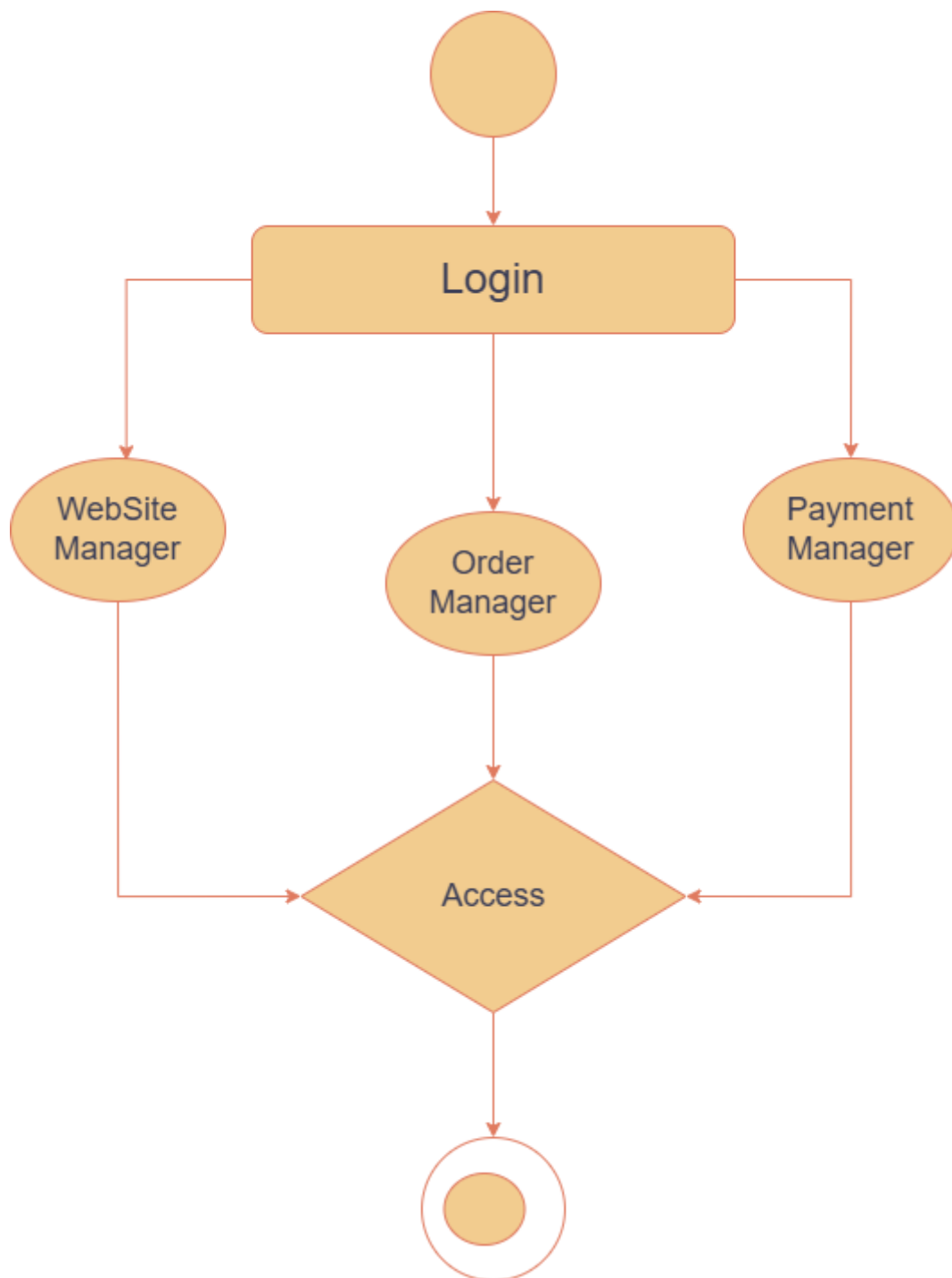
It can also be installed **locally** using **Docker** or **Node.js**, providing a controlled environment **for testing automation scripts**.

### Use Case Diagram:-



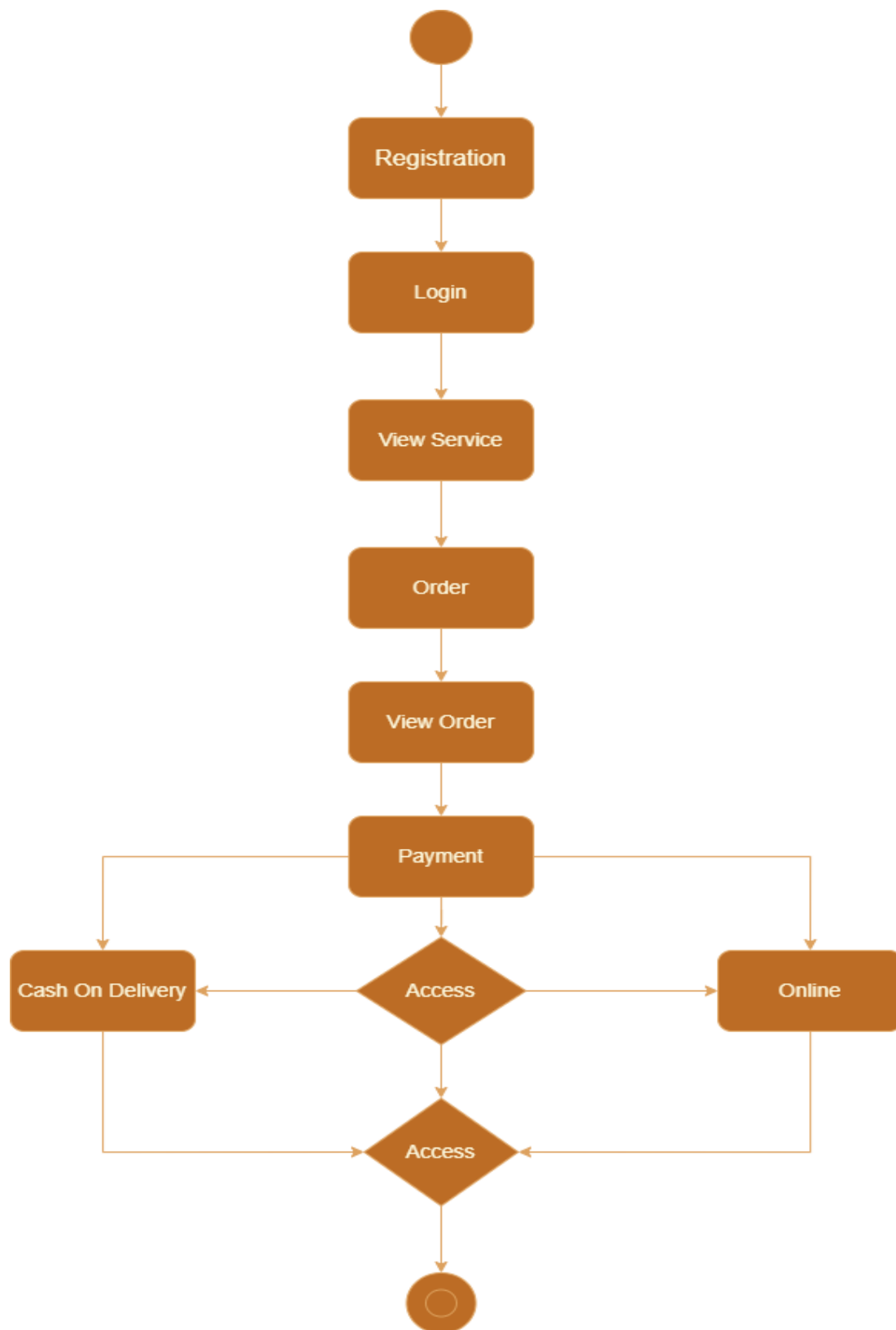
Use Case Diagram

**Activity Diagram:-**



**Activity Diagram for (admin)**

### Activity Diagram:-



Activity Diagram for (customer)

## Data Flow Diagram



DFD LEVEL 0

## Admin Information

| Field Name        | Data Type | Constraints |
|-------------------|-----------|-------------|
| <b>Admin_id</b>   | int       | Primary Key |
| <b>First_Name</b> | varchar   | Not Null    |
| <b>Last_Name</b>  | varchar   | Not Null    |
| <b>Password</b>   | int       | -           |
| <b>Email_id</b>   | varchar   | Not Null    |

### Customer Information

| Field Name  | Data Type | Constraints |
|-------------|-----------|-------------|
| Customer_id | int       | Primary Key |
| First_Name  | varchar   | Not Null    |
| Last_Name   | varchar   | Not Null    |
| Password    | int       | -           |
| Email_id    | varchar   | Not Null    |
| Address     | varchar   | -           |

### Service Information

| Field Name   | Data Type | Constraints |
|--------------|-----------|-------------|
| Service_id   | int       | Primary Key |
| Service_Name | varchar   | Not Null    |

**Order Details**

| Field Name  | Data Type | Constraints |
|-------------|-----------|-------------|
| Order_id    | int       | Primary Key |
| Quantity    | int       | Not Null    |
| Bill_id     | int       | Foreign Key |
| Customer_id | int       | Foreign Key |

Payment Details

| Field Name  | Data Type | Constraints |
|-------------|-----------|-------------|
| Bill_id     | int       | Primary Key |
| Customer_id | int       | Foreign Key |
| Bill_amount | int       | Not Null    |
| Order_id    | int       | Foreign Key |

## JuiceShopAutomationFramework

### OWASP-JuiceShop-Automation

```
|
|
|  └─ pom.xml
|
|  └─ testng.xml
|
|
|  └─ src/test/java
|
|    └─ base
|
|      └─ BaseClass.java
|
|
|
|    └─ pages
|
|      └─ LoginPage.java
|
|      └─ HomePage.java
|
|      └─ SearchPage.java
|
|
|
|    └─ tests
|
|      └─ LoginTest.java
|
|      └─ SearchTest.java
|
|      └─ AddToCartTest.java
|
|
|    └─ utils
|
|      └─ ExcelUtils.java
|
|      └─ ScreenshotUtils.java
|
|      └─ ConfigReader.java
```

- |
- |└─ **src/test/resources**
- | |└─ **config.properties**
- | └─ **TestData.xlsx**
- |
- |└─ **Reports**
- |└─ **Screenshots**
- └─ **README.md**

## **PRACTICAL 1: Maven Project Setup**

### **Step 1: Create Maven Project**

- Eclipse → File → New → Maven Project
- Artifact Id: OWASP-JuiceShop-Automation
- Packaging: jar

### **Step 2: pom.xml (MANDATORY PRACTICAL)**

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>automation</groupId>

  <artifactId>OWASP-JuiceShop-Automation</artifactId>

  <version>0.0.1-SNAPSHOT</version>

  <dependencies>

    <!-- Selenium -->

    <dependency>

      <groupId>org.seleniumhq.selenium</groupId>

      <artifactId>selenium-java</artifactId>

      <version>4.16.1</version>

    </dependency>
```

**<!-- TestNG -->**

**<dependency>**

**<groupId>org.testng</groupId>**

**<artifactId>testng</artifactId>**

**<version>7.9.0</version>**

**<scope>test</scope>**

**</dependency>**

**<!-- WebDriverManager -->**

**<dependency>**

**<groupId>io.github.bonigarcia</groupId>**

**<artifactId>webdrivermanager</artifactId>**

**<version>5.6.3</version>**

**</dependency>**

**<!-- Apache POI -->**

**<dependency>**

**<groupId>org.apache.poi</groupId>**

**<artifactId>poi-ooxml</artifactId>**

**<version>5.2.5</version>**

**</dependency>**

**</dependencies>**

**</project>**

## **PRACTICAL 2: BaseClass (Browser Launch)**

➤ **base/BaseClass.java**

```
package base;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

import io.github.bonigarcia.wdm.WebDriverManager;

public class BaseClass {

    public WebDriver driver;

    @BeforeMethod
    public void setup() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://demo.owasp-juice.shop/");
    }
}
```

**@AfterMethod**

**public void tearDown() {**

**driver.quit();**

**}**

**}**

### **PRACTICAL 3: Login Page (POM)**

➤ pages/LoginPage.java

```
package pages;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
public class LoginPage {
```

```
    WebDriver driver;
```

```
    By accountBtn = By.id("navbarAccount");
```

```
    By loginBtn = By.id("navbarLoginButton");
```

```
    By email = By.id("email");
```

```
    By password = By.id("password");
```

```
    By submit = By.id("loginButton");
```

```
    public LoginPage(WebDriver driver) {
```

```
        this.driver = driver;
```

```
    }
```

```
    public void login(String user, String pass) {
```

```
        driver.findElement(accountBtn).click();
```

```
        driver.findElement(loginBtn).click();
```

```
    driver.findElement(email).sendKeys(user);  
    driver.findElement(password).sendKeys(pass);  
    driver.findElement(submit).click();  
}  
}
```

## **PRACTICAL 4: Login Test Case**

➤ tests/LoginTest.java

```
package tests;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.Test;
```

```
import base.BaseClass;
```

```
import pages.LoginPage;
```

```
public class LoginTest extends BaseClass {
```

```
    @Test
```

```
    public void validLoginTest() throws InterruptedException {
```

```
        LoginPage lp = new LoginPage(driver);
```

```
        lp.login("test@test.com", "test123");
```

```
        Thread.sleep(3000);
```

```
        Assert.assertTrue(driver.getTitle().contains("OWASP"));
```

```
    }
```

```
}
```

## **PRACTICAL 5: Search Product**

➤ **pages/SearchPage.java**

```
package pages;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
public class SearchPage {
```

```
    WebDriver driver;
```

```
    By searchBox = By.id("searchQuery");
```

```
    public SearchPage(WebDriver driver) {
```

```
        this.driver = driver;
```

```
    }
```

```
    public void searchProduct(String product) {
```

```
        driver.findElement(searchBox).sendKeys(product);
```

```
    }
```

```
}
```

➤ tests/SearchTest.java

```
package tests;
```

```
import org.testng.annotations.Test;
```

```
import base.BaseClass;
```

```
import pages.SearchPage;
```

```
public class SearchTest extends BaseClass {
```

```
    @Test
```

```
    public void searchTest() throws InterruptedException {
```

```
        SearchPage sp = new SearchPage(driver);
```

```
        sp.searchProduct("Apple");
```

```
        Thread.sleep(3000);
```

```
    }
```

```
}
```

## **PRACTICAL 6: Add To Cart**

➤ tests/AddToCartTest.java

```
package tests;
```

```
import org.openqa.selenium.By;
```

```
import org.testng.annotations.Test;
```

```
import base.BaseClass;
```

```
public class AddToCartTest extends BaseClass {
```

```
    @Test
```

```
    public void addToCart() throws InterruptedException {
```

```
        driver.findElement(By.cssSelector("button[aria-label='Add to  
Basket']")).click();
```

```
        Thread.sleep(3000);
```

```
    }
```

```
}
```

## **PRACTICAL 7: TestNG XML**

➤ testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="OWASP Automation Suite">
```

```
  <test name="Smoke Test">
```

```
    <classes>
```

```
      <class name="tests.LoginTest"/>
```

```
      <class name="tests.SearchTest"/>
```

```
      <class name="tests.AddToCartTest"/>
```

```
    </classes>
```

```
  </test>
```

```
</suite>
```

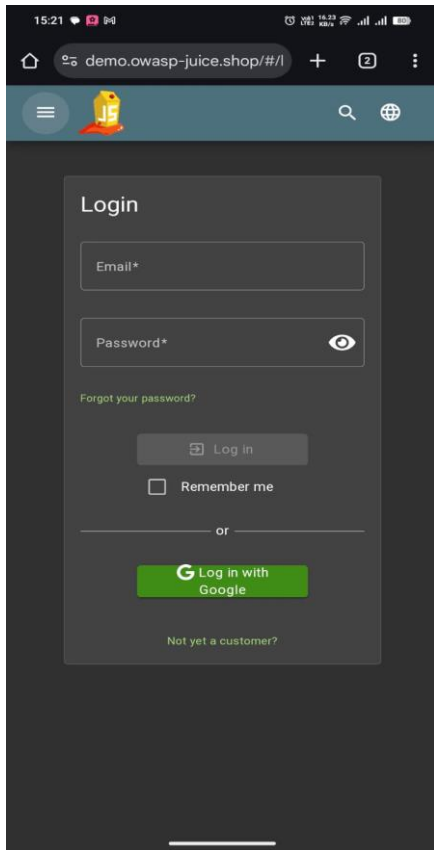
## **PRACTICAL 8: Config File**

➤ **config.properties**

**url=https://demo.owasp-juice.shop/**

**browser=chrome**

## Implementation



15:21 100% demo.owasp-juice.shop/#/l

Menu Search

### Login

Email\*

Password\* 

[Forgot your password?](#)

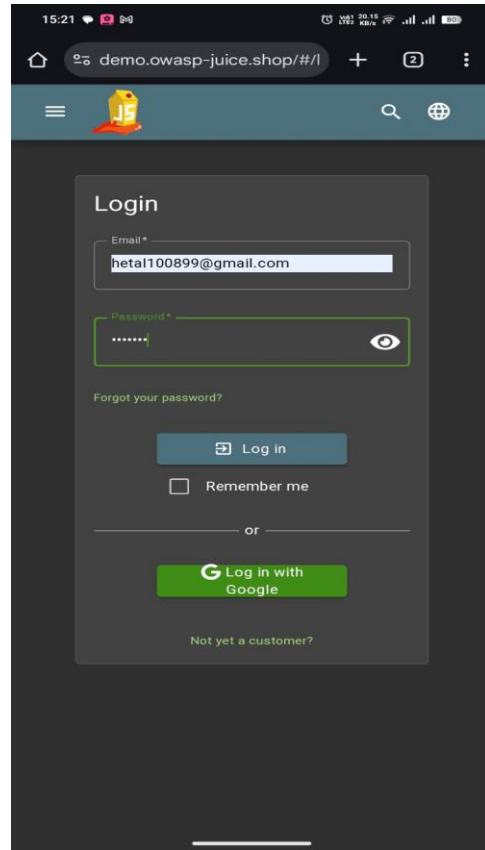
 Log in

☐ Remember me

or

 Log in with Google

[Not yet a customer?](#)



15:21 100% demo.owasp-juice.shop/#/l

Menu Search

### Login

Email\*  
hetal100899@gmail.com

Password\* 

[Forgot your password?](#)

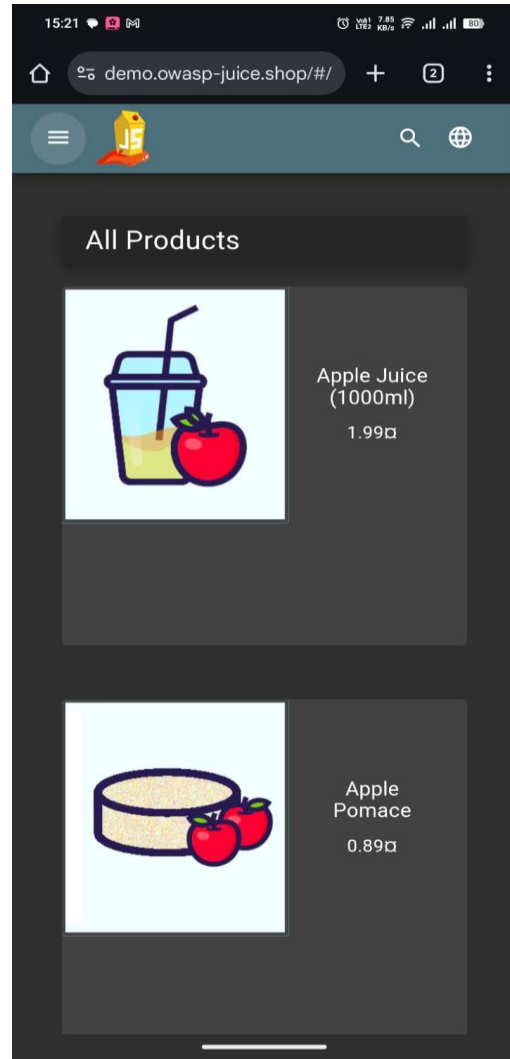
 Log in

☐ Remember me

or

 Log in with Google

[Not yet a customer?](#)



## Conclusion

The Automation Test Script Project on **OWASP Juice Shop** demonstrates the practical application of **automation testing using Selenium WebDriver, TestNG, and Java**. By developing a **hybrid automation framework** with modular design, data-driven capabilities, and reusable components, the project achieves efficient and reliable testing of core functionalities such as **Login, Signup, Product Search, Cart, and Checkout**.

Key takeaways from this project:

1. Automation reduces **manual effort**, improves **accuracy**, and ensures **reproducibility** of tests.
2. The use of **Page Object Model (POM)** ensures **maintainability and scalability** of the test scripts.
3. **Data-driven testing** allows execution of multiple test scenarios with varied inputs.
4. Reports, logs, and screenshots provide a clear **overview of test execution and failures**, supporting better decision-making.
5. The project serves as a **practical learning platform** for both functional testing and understanding web application behaviour.